Dynamic Programming

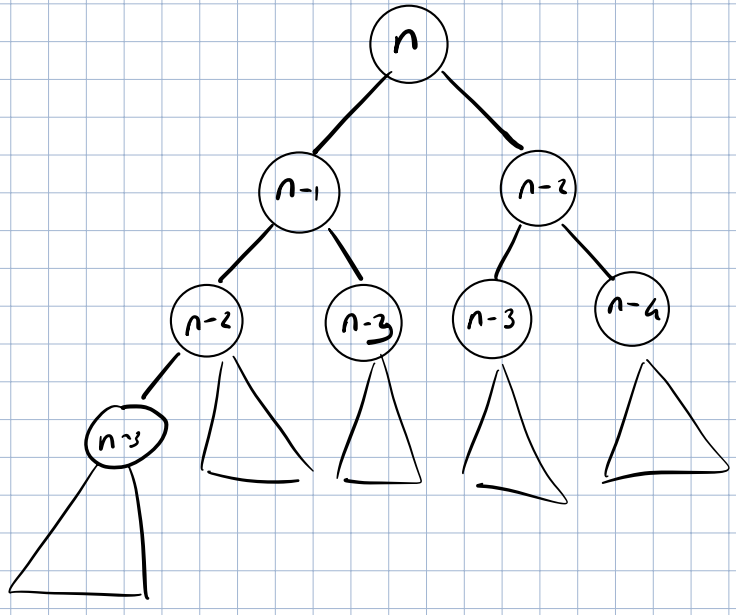
Fibonacci Numbers

$F_0 = 0$

$F_1 = 1$

$F_n = F_{n-1} + F_{n-2}$

```
def Fib(n):
    if n == 0
        return 0
    if n == 1
        return 1
    return Fib(n-1) + Fib(n-2)
```

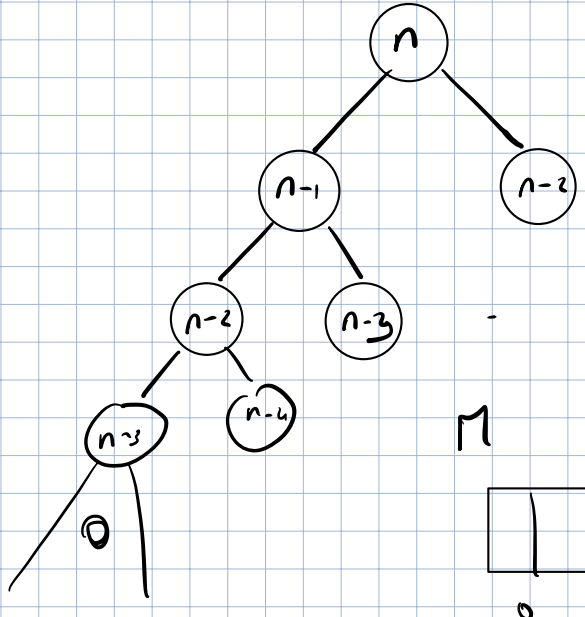


Idea: Memoization

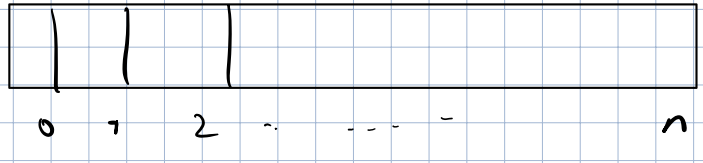
Store the Answers

Top - Down approach

```
M = [None] * (n+1)
def FibMemo(n, M):
    if n == 0
        return 0
    if n == 1
        return 1
    if M[n] == None:
        M[n] = FibMemo(n-1, M) +
               FibMemo(n-2, M)
    return M[n]
```



M



B. Bottom-Up Approach

```
def FibBU(n):
    M = [0] * [n+1]
    M[0] = 0
    M[1] = 1
    for i in range(2, n):
        M[i] = M[i-1] + M[i-2]
    return M
```

Faster Algorithm For n-th Fibonacci Number?

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \quad \begin{matrix} F_2 & F_1 \\ \| & \| \\ F_1 & F_0 \end{matrix}$$

$$A^n = \begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix}$$

(Prove $\underline{A^{n-1} \cdot A}$)

Matrix exponentiation

How to compute A^n?

n even $\quad A^n = A^{\frac{n}{2}} \cdot A^{\frac{n}{2}}$

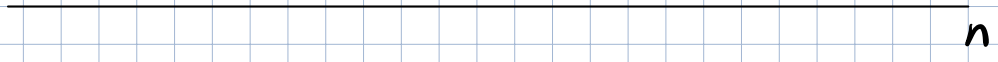
n odd $\quad A^n = A^{\frac{n}{2}} \cdot A^{\frac{n}{2}} \cdot A$

$\Theta(\log n)$ time

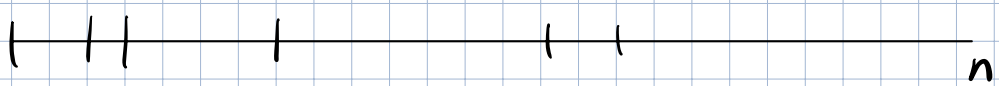
Rod Cutting

Steel rod of length n

For a cut of length $i \in [1, n]$, you get p_i euros



Goal: is to find the cuts that maximize the total revenue



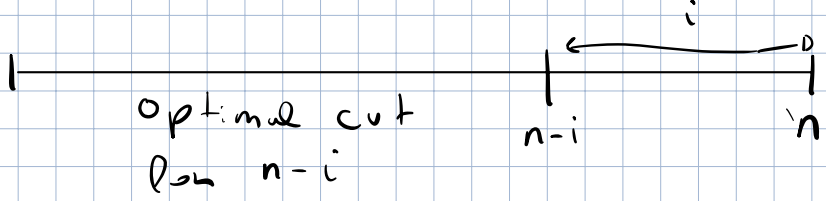
$p_2 + p_1 + p_4 + p_1 + p_2 + p_3 = R$

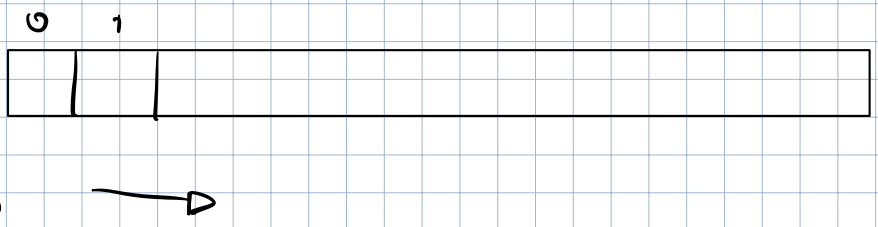
Solution

Subproblems: Optimal way to cut a rod of length $k < n$

Combine:

$$R(n) = \max_{1 \le i \le n} \{R(n-i) + p_i\}$$





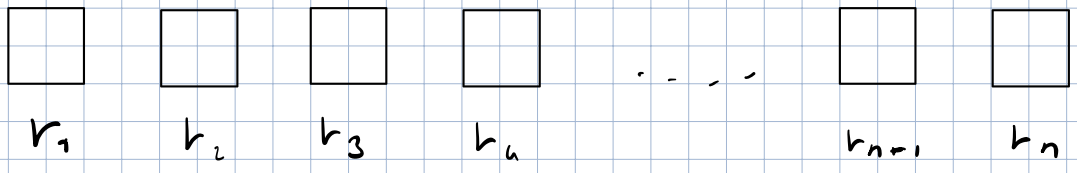

```
def rod(n):
    R = [0] * (n+1)
    for i in range(n+1):
        for e in range(i):
            R[i] = max(R[i],
                       R[i-e] + P[e])
```

$\Theta(n^2)$ time

Facebook Robber

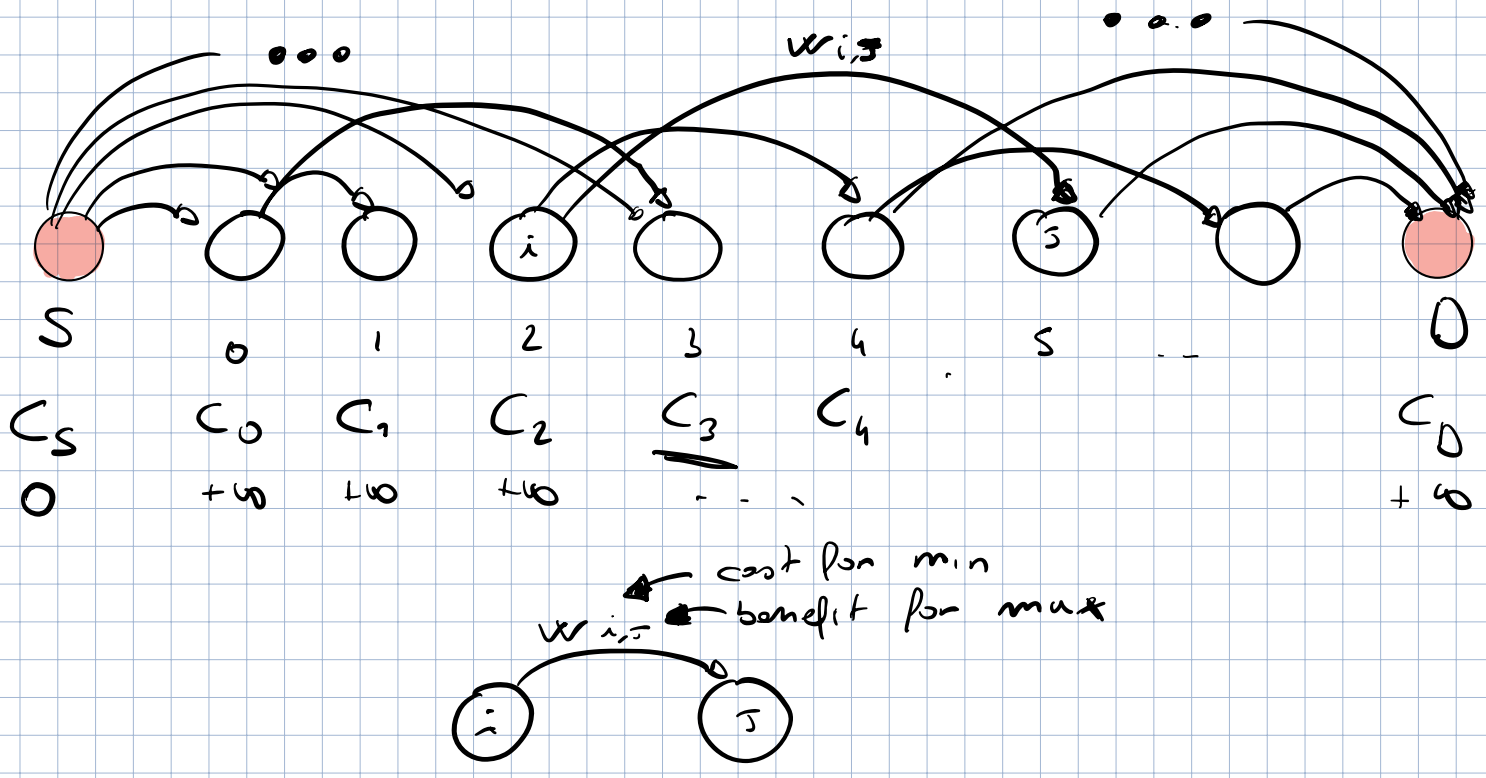
DON'T TRY THIS AT HOME

n houses



CONSTRAINT: if you steal in house i,
you cannot do it in houses
$i-1$ and $i+1$

Shortest / Longest Path computation on a DAG



while processing i,

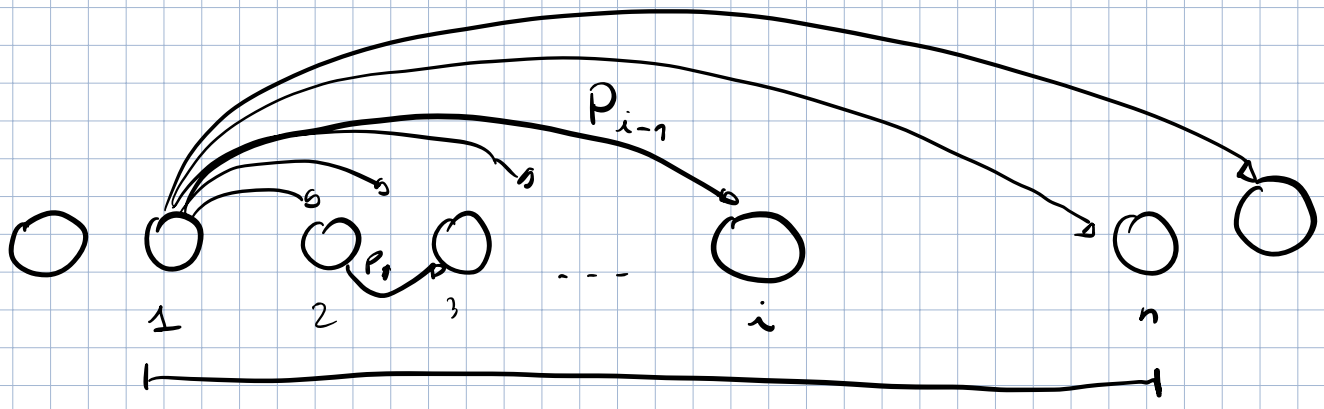
if $(i,j) \in E$, $C_j = \min\{C_j, C_i + w_{i,j}\}$

in case of longest path

if $(i,j) \in E$, $C_j = \max\{C_j, C_i + w_{i,j}\}$

$\Theta(|V| + |E|)$ time

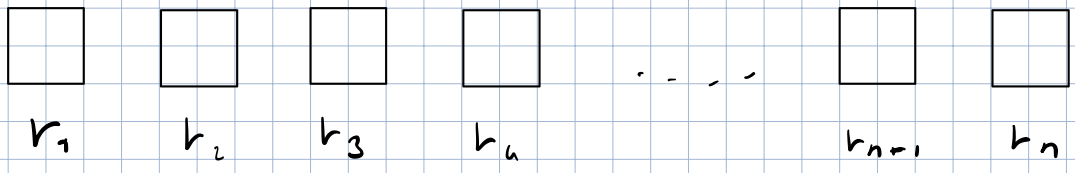
Rod cutting



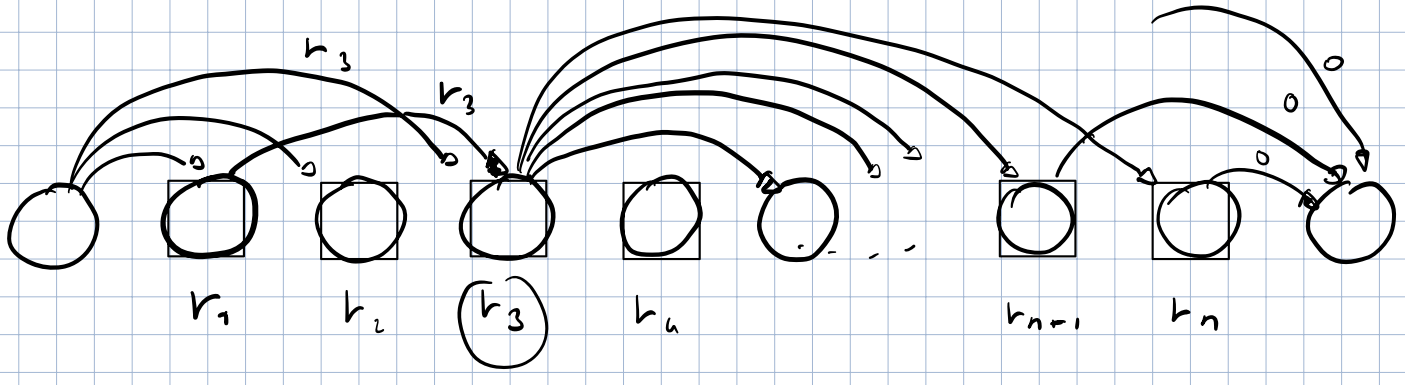
Facebook Robber

DON'T TRY THIS AT HOME

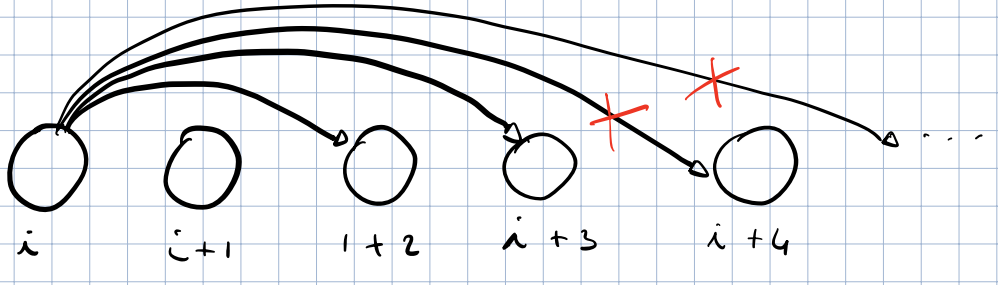
n houses



CONSTRAINT: If you steal in house i, you cannot do it in houses $i-1$ and $i+1$

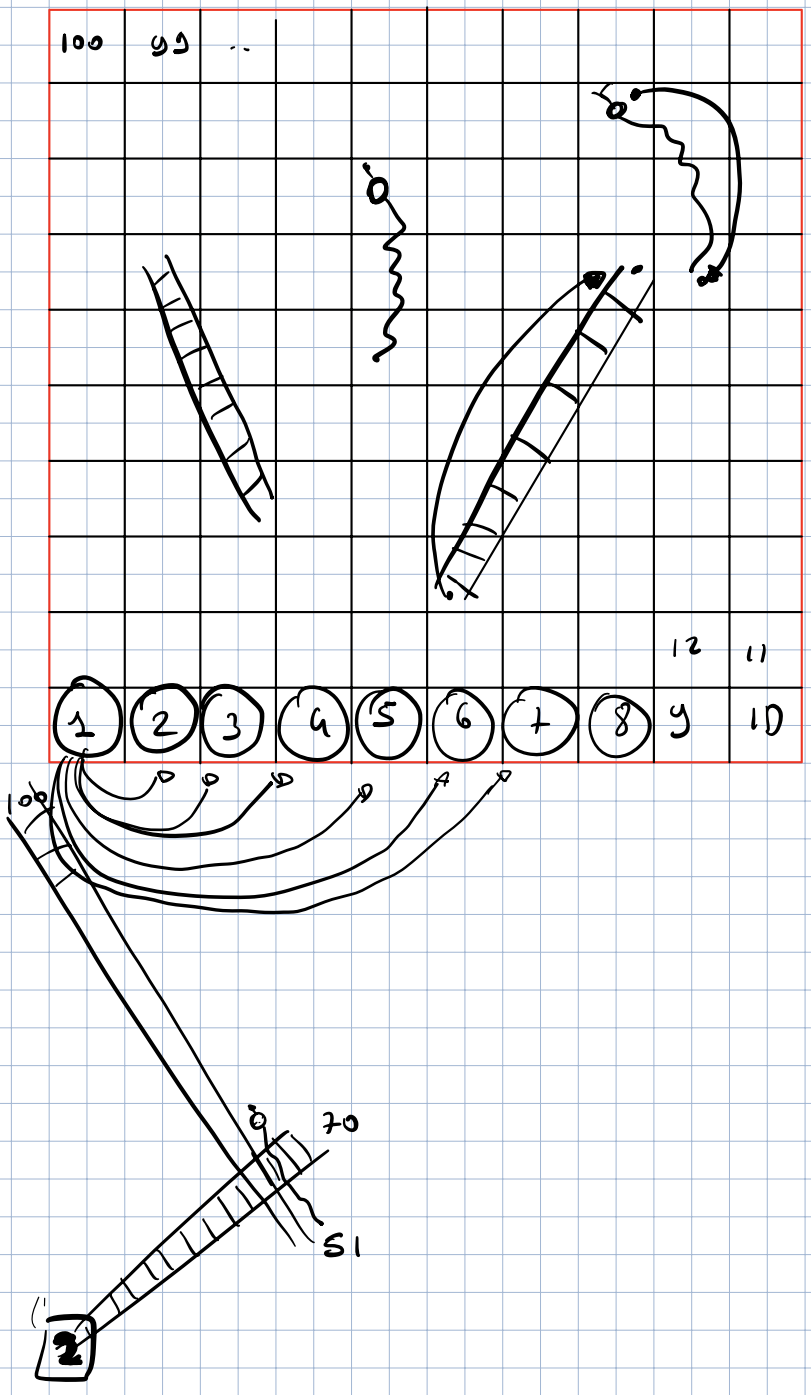


$\Theta(n^2)$ time



$\Theta(n)$ time

Snakes and Ladders



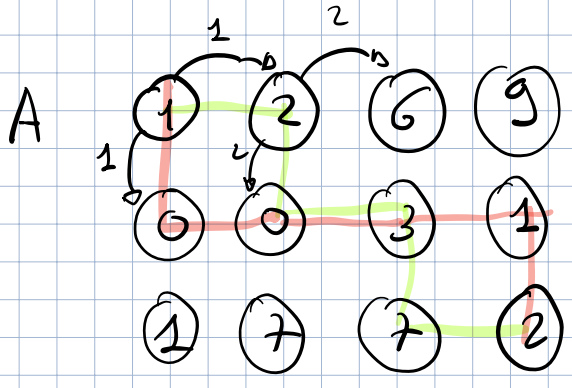
$\Theta(n)$ time

Minimum Cost Path

A is a $n \times m$ matrix with integers

The goal is to compute the path of minimal cost from the Top-left corner to the bottom right moving only down ↓ or right →

Example



1 + 2 + 0 + 3 + 7 + 2 = 15

1 + 0 + 0 + 3 + 1 + 2 = 7

Longest Common Subsequence (LCS)

Given S_1 ($|S_1| = n$) and S_2 ($|S_2| = m$)

Find $LCS(S_1, S_2)$

S_1 A B C B D A B $\quad n = 7$

S_2 B D C A B A $\quad m = 6$

Solution $\Theta(n \cdot m)$ time

we want to compute $LCS(S_1, S_2)$

Sub problems

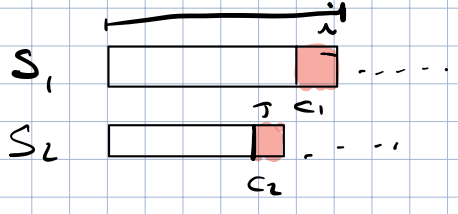
we want to compute LCS for prefixes of S_1 and S_2

$LCS(i, j)$ to denote the length of LCS between

$S_1[1 .. i]$ and $S_2[1 .. j]$

Combine

$LCS(i, j)$



$LCS(i, j) =$

CASE #1

if $c_1 == c_2$

$LCS(i, j) = 1 + LCS(i-1, j-1)$

CASE #2

if $c_1 \neq c_2$ max {

$LCS(i, j) = LCS(i, j-1),$

$LCS(i-1, j)$ }

~~$LCS(i-1, j-1)$~~

Recurrence

$$LCS(i,j) = \begin{cases} 0 & \text{if } i==0 \text{ or } j==0 \\ 1 + LCS(i-1, j-1) & \text{if } S[i]==S[j] \\ \max\{ LCS(i, j-1), LCS(i-1, j) \} & \text{otherwise} \end{cases}$$

Bottom_up Approach

		0	1	2	3	4	5	6
		∅	B	D	C	A	B	A
0	∅	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

$$LCS(i,j) = \begin{cases} 0 & \text{if } i==0 \text{ or } j==0 \\ 1 + LCS(i-1, j-1) & \text{if } S[i]==S[j] \\ \max\{ LCS(i, j-1), LCS(i-1, j) \} & \text{otherwise} \end{cases}$$

0/1 knapsack problem

We have n items and a capacity C

Each item i has a value v_i and a weigth w_i.

Goal: Select a subset of items that maximeses the total value subset to capacity C

Example $C = 7$

weigth	value	
1	1	$1 + 7 = 8$
3	4	$4 + 5 = 9$
4	5	
5	7	

Solution $\Theta(n \cdot C)$ time

Subproblems Fewer items and any capacity $J \leq C$

Fix an ordering of items and solve for a prefix of items

Combine $K(i, J)$ is an optimal selection for prefix up to ith item with capacity $J \leq C$

Do we want to include item i?

$$K(i, J) = \begin{cases} \max\{ K(i-1, J-w_i) + v_i, & \text{yes} \\ K(i-1, J) \} & \text{no} \end{cases}$$

Recurrence

$$K(i, J) = \begin{cases} 0 & \text{if } i == 0 \text{ or } J == 0 \\ \max\{ v_i + K(i-1, J-w_i), & J \geq 0 \\ K(i-1, J) \end{cases}$$

$$\| \quad K[i, J]$$

$w_i (=1)$

w	v	i	0	1	2	3	4	5	6	7
/	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	1	1	1	1	1	1
3	4	2	0	1	1	4	5	5	5	5
4	5	3	0	1	1	4	5	6	6	9
5	7	4	0	1	1	4	5	7	8	9

Fractional Knapsack problem

You can select fraction of items

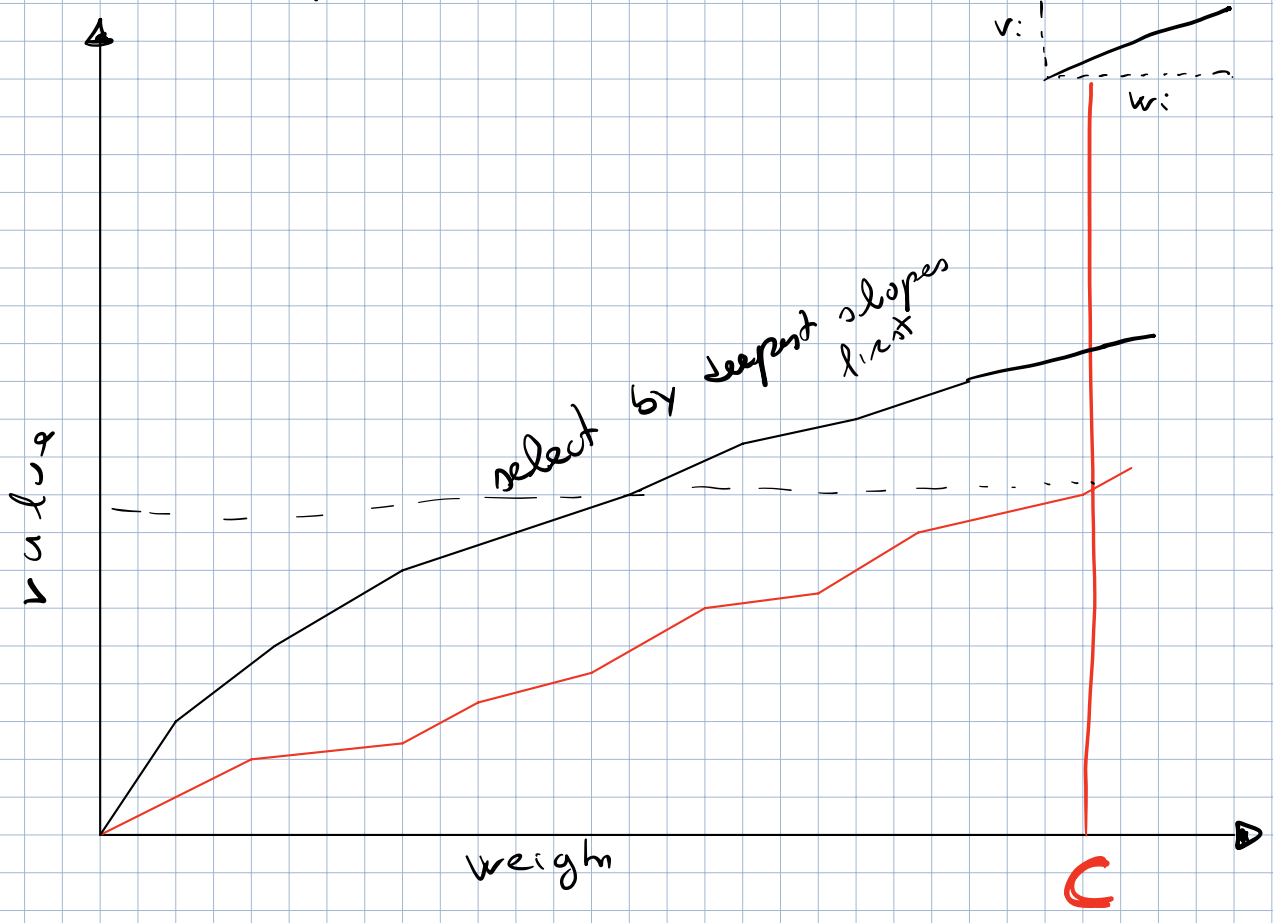
$0 \leq f_i \leq 1$ costs $w_i \cdot f_i$
and value $v_i \cdot f_i$

Optimal solution in $\Theta(n \log n)$ time

Greedy Solution

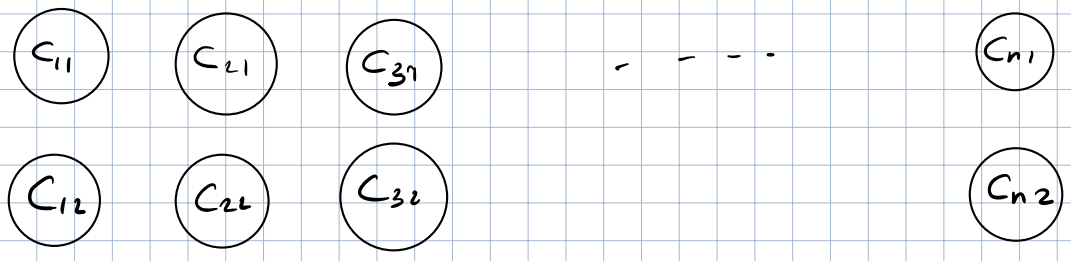
$ratio_i = \frac{v_i}{w_i}$ value per unit of weight

- Sort items by decreasing ratios
  select items until we ran out of capacity



Puzzle / Exercise

n pairs coins



Select k coins to maximize the total value, but you can select the second coin of a pair only if also the first one is selected.

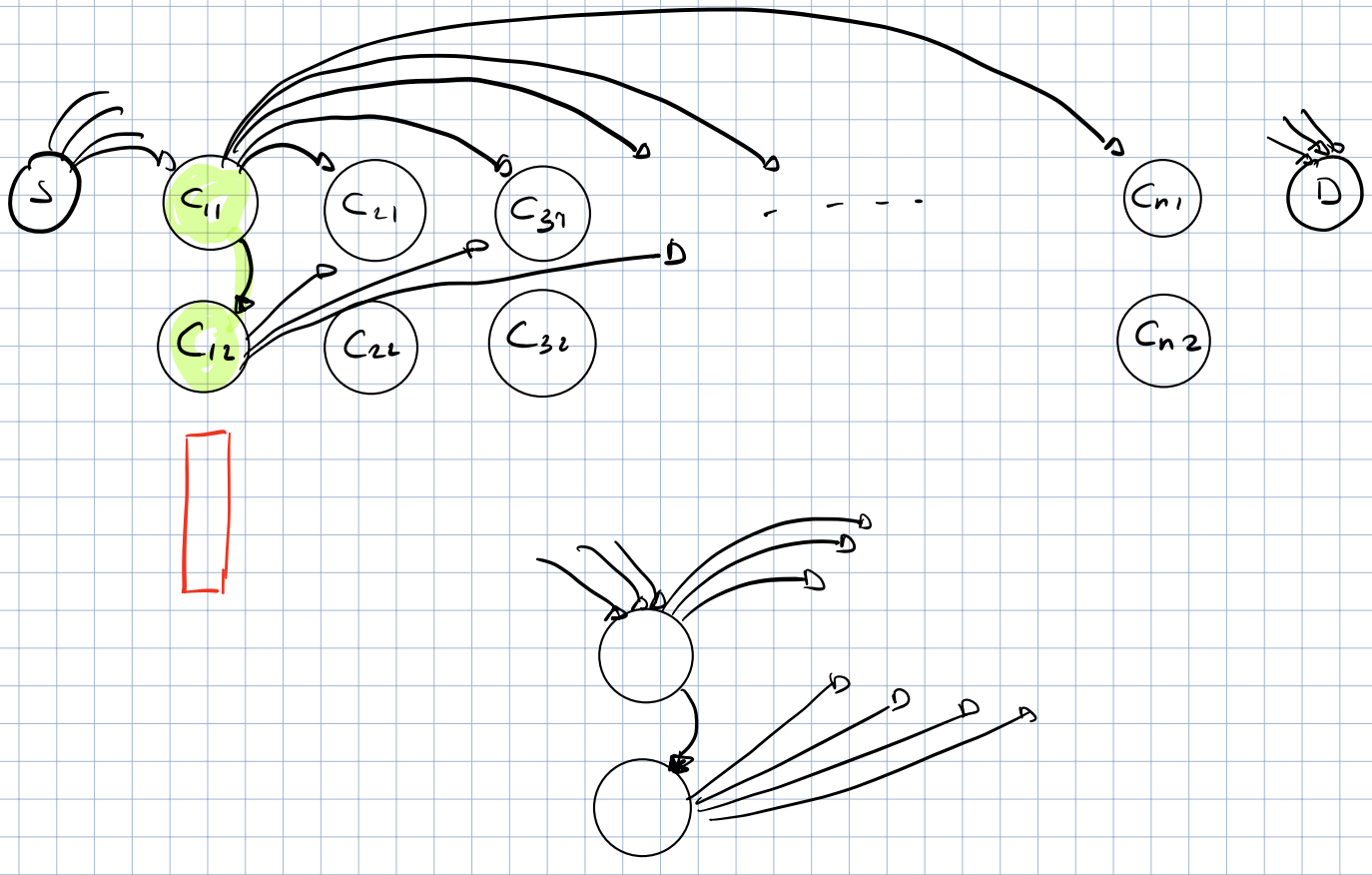
(10)	(1)	3
1	(12)	3

$k = 1$ 10

$k = 2$ $10 + 3 = 13$

$k = 3$ $10 + 1 + 12 = 23$

Solution #1 $\Theta(n^2)$ time



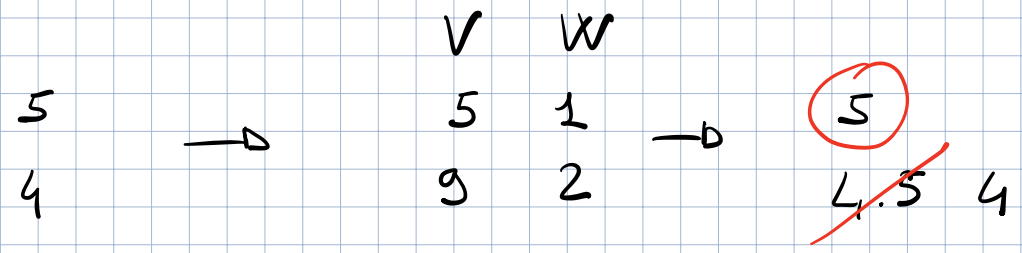
Longest Path on this in $\Theta(n^2)$ time

Solution #2 $\Theta(kn)$ time

V	W
$c_{1,2}$	1
$c_{1,1}+c_{1,2}$	2
$c_{2,2}$	1
$c_{2,1}+c_{2,2}$	2
⋮	

it runs in $\Theta(n \cdot 4)$ time

Solution # 3 $\Theta(n \log n)$ time



Longest Increasing Subsequence (LIS)

Given a sequence S of n elements, find the LIS of S

S	10	22	9	21	33	50	41	60	80
IS			-		↑	-	-	-	

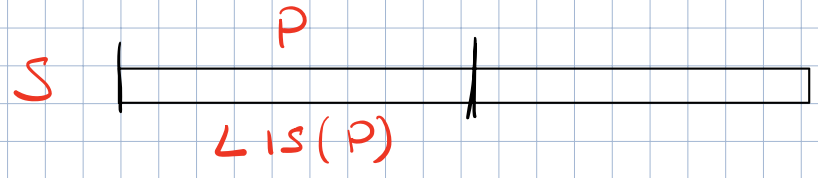
increasing subsequence

LIS - - - - - - -

LIS - - - - - - -

Solution #1 $\Theta(n^2)$ time

Subproblem



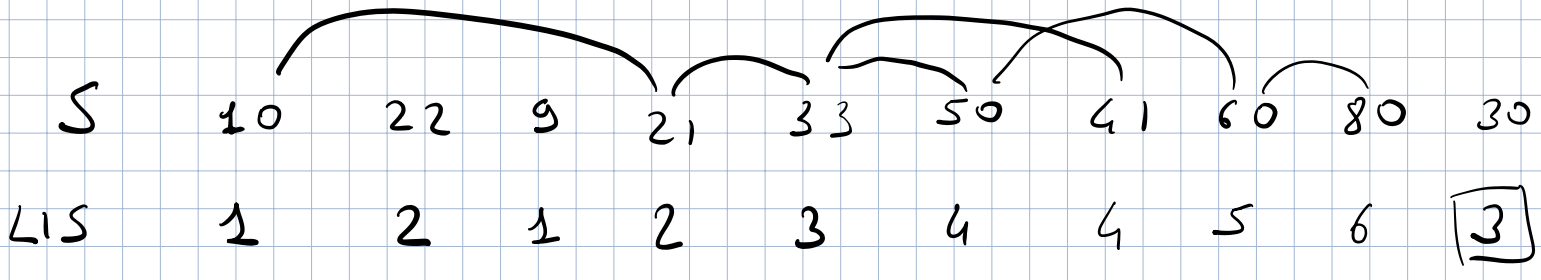
1 2 10 100 | 0 0 0 ...

len (LIS(P)) = 4

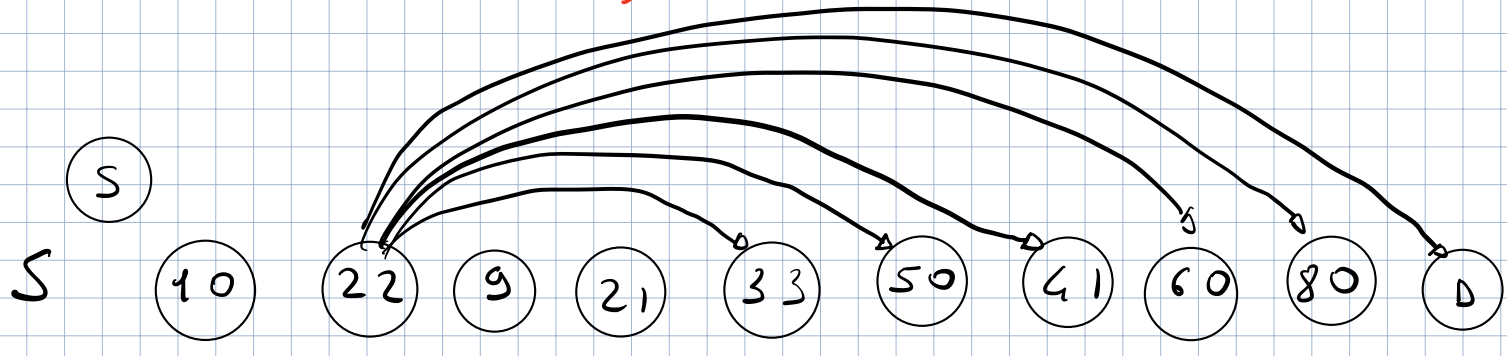
LIS(i) = is the LIS of S[0 ... i] which ends with S[i]

Recurrence

$$LIS(i) = 1 + \max \{ LIS[j] \mid 1 \le j \le i-1 \text{ AND } S[j] < S[i] \}$$



Solution #2 $\Theta(n^2)$ time



Longest path in $\Theta(n^2)$ time

LIS in $\Theta(n \log n)$ time

we say that a position i dominates a position j

if $S[i] < S[j]$ AND

$LIS[i] \geq LIS[j]$

		i		j		
S	...	10	...	15	...	16 ... 13
LIS	...	2	...	2	...	

S	10	22	9	21	33	50	41	60	80
LIS	1	2	1	2	3	4	4	5	6

All the values in LIS are represented

there is exactly one dominating position for each possible value of LIS

S	10	22	9	21	33	50	41	60	80
LIS	1	2	1	2	3	4	4	5	6

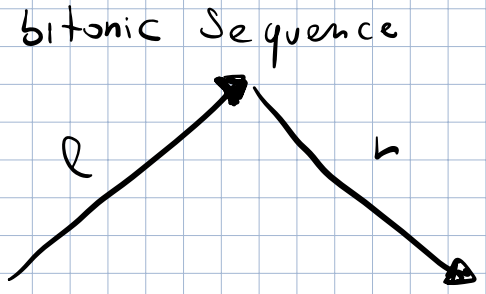
D

LIS	1	2	3	4	5	6
V	9	21	33	41	60	80

↑ dominating position

n predecessor queries in $\Theta(n \log n)$ time

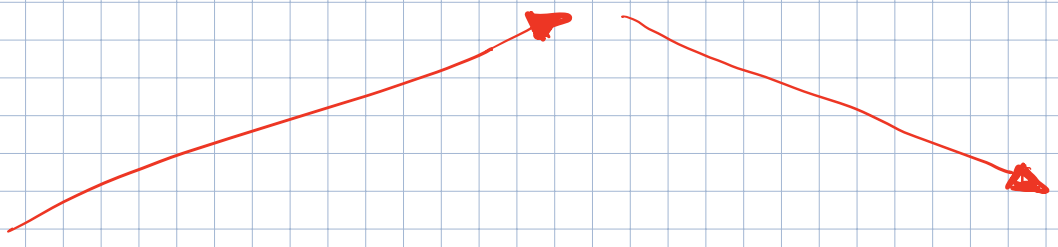
Longest Bitonic Subsequence (LBS)



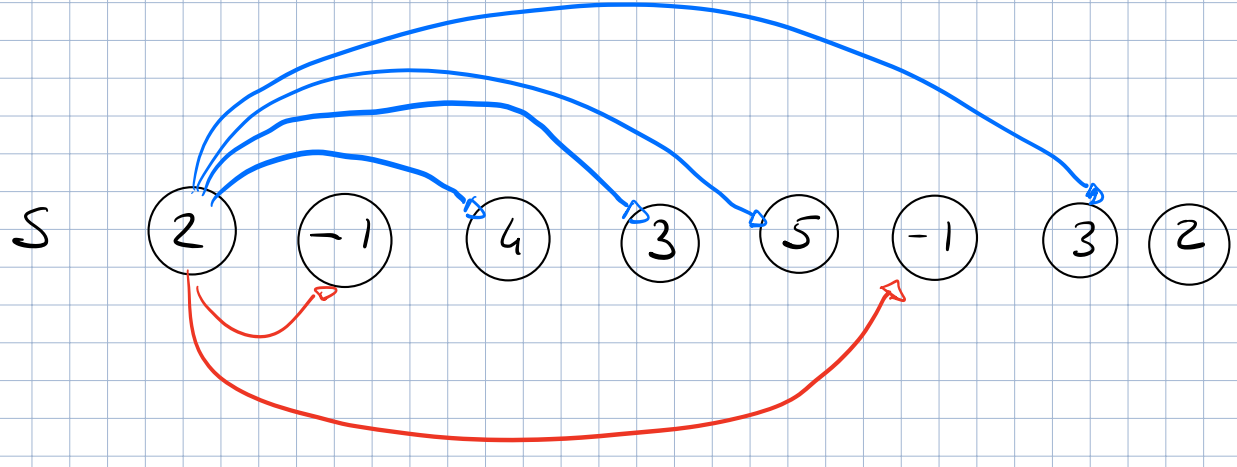
l and r may have different values and may be 0

Example

5 2 -1 4 3 5 -1 3 2



Solution in $\Theta(n^2)$ time



Solution in $O(n \log n)$ time